🚨 PAN!C - COMPLETE SYSTEM WALKTHROUGH & FLOW

FOR: Understanding architecture, data flow, and dynamic user input
AUDIENCE: Developers wanting to understand how everything connects
TIME: 30 minutes to understand the full system

TABLE OF CONTENTS

1. **System Architecture Overview**
2. **How Data Flows (User Input to Backend to Database)**
3. **The Complete User Journey (Step-by-Step)**
4. **Database Schema & Why It's Not Hardcoded**
5. **API Endpoints Explained**
6. **Frontend to Backend Communication**
7. **Firebase Cloud Messaging Flow**
8. **Real Example: User Presses Panic Button**
9. **Why Nothing is Hardcoded**
10. **Testing the Complete Flow Locally**

---

1. SYSTEM ARCHITECTURE OVERVIEW

```
┌─────────────────────────────────────────────────────────────┐
│                    USER'S DEVICE (Frontend)                 │
│                                                             │
│   ┌─────────────────────────────────────────────────────┐   │
│   │ React Native App                                    │   │
│   │ ├─ Login/Signup Screen (User inputs email, password)  │   │
│   │ ├─ Home Dashboard (Shows user's data from DB)       │   │
│   │ ├─ Contacts Manager (User adds/edits contacts)      │   │
│   │ ├─ Documents Vault (User uploads docs)              │   │
│   │ ├─ Check-In Timer (User sets intervals)             │   │
│   │ └─ Panic Button (User presses → sends to backend)   │   │
│   └─────────────────────────────────────────────────────┘   │
│                                                             │
│                         ↕ (API Calls over HTTPS)            │
└─────────────────────────────────────────────────────────────┘


┌─────────────────────────────────────────────────────────────┐
│                   REPLIT SERVER (Backend)                   │
```

```
│   ┌──────────────────────────────────────────────────────────┐   │
│   │ Node.js + Express                                        │   │
│   │ ├─ /api/auth/* (signup, login, logout)                   │   │
│   │ ├─ /api/contacts/* (CRUD contacts)                       │   │
│   │ ├─ /api/documents/* (upload, encrypt, store)             │   │
│   │ ├─ /api/checkin/* (timer, history, snooze)               │   │
│   │ ├─ /api/panic/* (trigger, disarm)                        │   │
│   │ └─ /api/chat/* (AI responses)                            │   │
│   └──────────────────────────────────────────────────────────┘   │
│                              ↕ (MongoDB Queries)                  │
└──────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────┐
│              MONGODB ATLAS (Database - Single Source of Truth)    │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │ Database: "panic"                                        │   │
│   │ ├─ users (email, fullName, phone, fcmToken, etc.)        │   │
│   │ ├─ contacts (userEmail, name, phone, fcmToken, etc.)     │   │
│   │ ├─ documents (userEmail, type, encrypted data)           │   │
│   │ ├─ checkInSettings (userEmail, interval, hours)          │   │
│   │ ├─ checkInHistory (userEmail, timestamp, location)       │   │
│   │ ├─ incidents (userEmail, location, contacts notified)    │   │
│   │ └─ chatbotConversations (userEmail, messages)            │   │
│   └──────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────┘


┌──────────────────────────────────────────────────────────────────┐
│         EXTERNAL SERVICES (Firebase, ElevenLabs, Gemini)          │
│ ├─ Firebase Cloud Messaging (push notifications)                  │
│ ├─ Google Gemini (AI chatbot)                                     │
│ ├─ ElevenLabs (audio generation)                                  │
│ └─ Solana (blockchain incident records)                           │
└──────────────────────────────────────────────────────────────────┘
```

---

2. HOW DATA FLOWS (User Input to Backend to Database)

Example: User Signs Up

```
STEP 1: USER ENTERS DATA (Frontend - React)

┌──────────────────────────────┐
│ Sign Up Screen               │
│ ├─ Email input: "alex@email.com" │
```

```
│ ├─ Password input: "secret123"   │
│ ├─ Full Name: "Alex Chen"        │
│ └─ Phone: "+1(555)123-4567"      │
└──────────────────────────────────┘
                ↓
STEP 2: SEND REQUEST (Frontend → Backend)
POST /api/auth/signup
{
  "email": "alex@email.com",
  "password": "secret123",
  "fullName": "Alex Chen",
  "phone": "+1(555)123-4567"
}
                ↓
STEP 3: BACKEND PROCESSES (Node.js)
1. Validate input (email format, password length)
2. Check if user already exists in MongoDB
3. Hash password with bcrypt (10 rounds)
4. Generate JWT tokens (access + refresh)
5. Create user record
                ↓
STEP 4: STORE IN DATABASE (MongoDB)
db.collection('users').insertOne({
  _id: ObjectId("507f1f77bcf86cd799439011"),
  email: "alex@email.com",
  password: "$2b$10$hashed...",        // Bcrypt hashed
  fullName: "Alex Chen",
  phone: "+1(555)123-4567",
  fcmToken: null,                      // Will be set after signup
  createdAt: ISODate("2026-05-09"),
  settings: {
    theme: "dark",
    language: "en",
    notifications: true
  }
})
                ↓
STEP 5: RETURN RESPONSE (Backend → Frontend)
{
  "user": {
    "_id": "507f1f77bcf86cd799439011",
    "email": "alex@email.com",
    "fullName": "Alex Chen"
  },
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
```

```
  "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
                    ↓
STEP 6: FRONTEND STORES TOKEN (React)
localStorage.setItem('authToken', token)
localStorage.setItem('userId', user._id)
```

KEY POINT: Nothing is hardcoded. User entered their own data. It's stored in MongoDB. Backend retrieved it. Frontend displays it.

---

3. THE COMPLETE USER JOURNEY (Step-by-Step)

Journey 1: New User Registration

```
Day 1 - User Opens App

┌──────────────────────────────────────────────────────────┐
│ SPLASH SCREEN                                            │
│ [Login]  [Sign Up]                                       │
└──────────────────────────────────────────────────────────┘

                          ↓
User clicks [Sign Up]

┌──────────────────────────────────────────────────────────┐
│ STEP 1 OF 4: CREATE ACCOUNT                              │
│ Email: [user types: alex@email.com]                     │
│ Password: [user types: secure_password_123]              │
│ Full Name: [user types: Alex Chen]                       │
│ Phone: [user types: +1(555)123-4567]                     │
│ [NEXT]                                                   │
└──────────────────────────────────────────────────────────┘

                          ↓
Backend creates user in MongoDB with entered data
                          ↓

┌──────────────────────────────────────────────────────────┐
│ STEP 2 OF 4: ADD FIRST CONTACT                           │
│ Contact Name: [user types: Maria Garcia]                 │
│ Phone: [user types: +1(555)987-6543]                     │
│ Email: [user types: maria@email.com]                     │
│ Relationship: [user selects: Mom]                        │
│ [NEXT]                                                   │
└──────────────────────────────────────────────────────────┘

                          ↓
Backend creates contact in MongoDB: contacts collection
```

```
                          ↓

┌──────────────────────────────────────────────────────────┐
│ STEP 3 OF 4: UPLOAD DOCUMENTS                            │
│ [Upload Passport] (user selects: passport.pdf)           │
│ Expiry Date: [user enters: 12/28/2027]                   │
│ [Upload Visa] (user selects: visa.pdf)                   │
│ Expiry Date: [user enters: 06/27/2028]                   │
│ [NEXT]                                                   │
└──────────────────────────────────────────────────────────┘

                          ↓
Documents encrypted client-side (AES-256-GCM)
Encrypted bytes + metadata stored in MongoDB
                          ↓

┌──────────────────────────────────────────────────────────┐
│ STEP 4 OF 4: SET SAFE PHRASE                             │
│ Safe Phrase: [user types: my_secret_phrase_2024]         │
│ Confirm: [user types: my_secret_phrase_2024]             │
│ [CREATE ACCOUNT]                                         │
└──────────────────────────────────────────────────────────┘

                          ↓
Safe phrase bcrypt hashed, stored in MongoDB
FCM token obtained and registered with backend
                          ↓

┌──────────────────────────────────────────────────────────┐
│ HOME DASHBOARD                                           │
│ Welcome, Alex! 👋                                        │
│ Check-In: 30 minutes remaining                           │
│ [🚨 PANIC] button available                              │
│ Quick Links: [Contacts] [Documents] [Settings] [Chat]    │
└──────────────────────────────────────────────────────────┘
```

Journey 2: Checking In (Daily)

```
Day 2 - User Checks In

┌──────────────────────────────────────────────────────────┐
│ HOME DASHBOARD                                           │
│ Check-In: 5 minutes remaining ⚠️                         │
│ [Check In Now] [Snooze 5 min]                            │
└──────────────────────────────────────────────────────────┘

                          ↓
User clicks [Check In Now]
                          ↓
Backend:
1. Gets current GPS location
```

```
2. Creates record in checkInHistory collection
3. Updates nextDueAt (now + 30 minutes)
4. Returns success
                        ↓

┌──────────────────────────────────────────────────────┐
│ ✅ Check-in recorded!                                 │
│ Last check-in: Just now                              │
│ Next due: 30 minutes from now                        │
└──────────────────────────────────────────────────────┘
```

Journey 3: Panic Situation (Real Emergency)

```
Day 3 - User Encounters ICE Agents

┌──────────────────────────────────────────────────────┐
│ HOME DASHBOARD                                       │
│ [🚨 PANIC] ← User presses button                     │
└──────────────────────────────────────────────────────┘

                        ↓
STEP 1: CAPTURE DATA (Frontend)
├─ Get GPS location (latitude, longitude, address)
├─ Get current time
└─ Get all user's emergency contacts from localStorage


                        ↓
STEP 2: SEND TO BACKEND
POST /api/panic/trigger
{
  "latitude": 38.5816,
  "longitude": -121.4944,
  "address": "123 Main St, Sacramento, CA"
}


                        ↓
STEP 3: BACKEND PROCESSING
1. Verify user is authenticated (JWT check)
2. Get user from MongoDB (alex@email.com)
3. Get all contacts from MongoDB for this user
4. Extract FCM tokens from contacts
5. Create incident record in MongoDB
6. Send Firebase Cloud Message to each contact


                        ↓
STEP 4: CREATE INCIDENT RECORD
db.collection('incidents').insertOne({
```

```
  _id: ObjectId("507f1f77bcf86cd799439012"),
  userEmail: "alex@email.com",
  userName: "Alex Chen",
  userPhone: "+1(555)123-4567",
  triggeredAt: ISODate("2026-05-09T14:30:00Z"),
  location: {
    latitude: 38.5816,
    longitude: -121.4944,
    address: "123 Main St, Sacramento, CA"
  },
  contactsNotified: [
    {
      contactId: ObjectId("507f1f77bcf86cd799439011"),
      contactName: "Maria Garcia",
      fcmToken: "eESTx2nRfFk:APA91bFvMg...",
      notifiedAt: ISODate("2026-05-09T14:30:01Z"),
      status: "sent"
    }
  ],
  status: "active",
  sentCount: 1,
  failedCount: 0
})
```

↓

```
STEP 5: SEND FIREBASE CLOUD MESSAGES
For each contact with FCM token:
  firebase.messaging().send({
    notification: {
      title: "🚨 Alex Chen NEEDS HELP!",
      body: "Location: 123 Main St, Sacramento, CA"
    },
    data: {
      incidentId: "507f1f77bcf86cd799439012",
      userName: "Alex Chen",
      address: "123 Main St, Sacramento, CA",
      latitude: "38.5816",
      longitude: "-121.4944"
    },
    token: "eESTx2nRfFk:APA91bFvMg..."
  })
```

↓

```
STEP 6: CONTACTS RECEIVE NOTIFICATION
```

Maria's phone (even if app is closed):

```
┌──────────────────────────────────────────────────────────┐
│ 🚨 Alex Chen NEEDS HELP!                                 │
│ Location: 123 Main St, Sacramento, CA                    │
│ [TAP TO VIEW]                                            │
└──────────────────────────────────────────────────────────┘
```

↓

STEP 7: CONTACT TAPS NOTIFICATION
Opens PAN!C app → Shows incident details:

```
┌──────────────────────────────────────────────────────────┐
│ EMERGENCY INCIDENT                                       │
│ Who: Alex Chen                                           │
│ When: Just now (2:30 PM)                                 │
│ Where: 123 Main St, Sacramento, CA                       │
│ [View Location on Map]                                   │
│ [View Documents]                                         │
│ Emergency Contacts: Maria Garcia, Carlos Lopez           │
│ Safe Phrase Setup: Yes                                   │
└──────────────────────────────────────────────────────────┘
```

↓

STEP 8: BACKGROUND - SOUND ALERT
Audio loop plays on Alex's phone:
"HELP ICE AGENTS HELP LA MIGRA"
(Repeating for 2 minutes or until disarmed)

↓

STEP 9: ALEX GETS TO SAFETY
Alex enters safe phrase to disarm panic:

```
┌──────────────────────────────────────────────────────────┐
│ PANIC ACTIVE                                             │
│ Safe Phrase: [••••••••••••]                              │
│ [SUBMIT]                                                 │
└──────────────────────────────────────────────────────────┘
```

Backend verifies safe phrase (bcrypt compare)
Updates incident: status = "disarmed"
Stops sending alerts
Audio stops

↓

STEP 10: INCIDENT LOGGED
MongoDB incident record:
{

```
  status: "disarmed",
  disarmedAt: ISODate("2026-05-09T14:35:00Z"),
  duration: "5 minutes",
  contactsNotified: 1,
  delivered: 1
}
```

---

4. DATABASE SCHEMA & WHY IT'S NOT HARDCODED

Why We Use a Database (Not Hardcoded)

Hardcoded Example (❌ BAD):

```javascript
// BAD: Hardcoded data
const emergencyContacts = [
  { name: "Maria Garcia", phone: "+1(555)987-6543" },
  { name: "Carlos Lopez", phone: "+1(555)123-4567" }
];

// Problem: Every user gets the same contacts!
// Problem: Can't add/edit/delete contacts
// Problem: Not secure
// Problem: Not scalable
```

Database Approach (✅ GOOD):

```javascript
// GOOD: User's own data in database
db.collection('contacts').find({ userEmail: "alex@email.com" })
// Returns: Only Alex's contacts (Maria, Carlos)

db.collection('contacts').find({ userEmail: "maria@email.com" })
// Returns: Only Maria's contacts (different from Alex!)
```

Complete Schema (Why Each Field Exists)

```typescript
// users Collection
{
  _id: ObjectId,
```

```
  email: string,           // ← User enters during signup
  password: string,        // ← User enters, bcrypt hashed
  fullName: string,        // ← User enters during signup
  phone: string,           // ← User enters during signup
  profilePhoto: Buffer,    // ← User uploads (optional)
  defaultLocation: string, // ← User sets in settings

  fcmToken: string,        // ← Device registers after signup
                           //   (Firebase Cloud Messaging token)

  createdAt: Date,         // ← Auto-set by server
  lastLogin: Date,         // ← Auto-updated on each login
  isActive: boolean,       // ← Set after signup (no email verification)

  settings: {
    theme: 'dark' | 'light',
    language: 'en' | 'es' | 'tl',
    notifications: boolean
  },

  safePhrase: string,      // ← User enters, bcrypt hashed
}

// contacts Collection
{
  _id: ObjectId,
  userEmail: string,       // ← Links to user (filter by userEmail)
  name: string,            // ← User enters
  phone: string,           // ← User enters
  email: string,           // ← User enters
  relationship: string,    // ← User selects

  fcmToken: string,        // ← Contact's device registers
                           //   (filled when contact signs up)

  notifyVia: {
    sms: boolean,          // ← User toggles
    email: boolean,        // ← User toggles
    push: boolean          // ← User toggles
  },

  canSeeDocuments: boolean, // ← User toggles
  canSeeLocation: boolean,  // ← User toggles
  isPrimary: boolean,       // ← User marks primary contact
```

```
  createdAt: Date,          // ← Auto-set
  lastNotified: Date        // ← Auto-updated on panic
}

// documents Collection
{
  _id: ObjectId,
  userEmail: string,        // ← Links to user
  type: string,             // ← User selects (passport, visa, id, etc.)
  fileName: string,         // ← From file user uploaded

  fileData: Buffer,         // ← Encrypted document content
  encryptionIV: Buffer,     // ← Random IV for encryption
  encryptionAuthTag: Buffer, // ← Auth tag for AES-256-GCM

  uploadedAt: Date,         // ← Auto-set
  expiresAt: Date,          // ← User enters optional expiry

  isSharedWithContacts: boolean, // ← User toggles
  shareableLink: string,    // ← Auto-generated unique link
  shareableExpiresAt: Date // ← Auto-set to 24 hours from now
}

// checkInSettings Collection
{
  _id: ObjectId,
  userEmail: string,        // ← Links to user

  intervalMinutes: number, // ← User selects (15, 30, 60, 120)

  activeHours: {
    start: string,          // ← User sets (e.g., "09:00")
    end: string             // ← User sets (e.g., "22:00")
  },

  reminders: [
    { minutesBefore: 5 },
    { minutesBefore: 1 }
  ],

  nextDueAt: Date,          // ← Auto-calculated
  lastCheckedInAt: Date,    // ← Auto-updated

  isActive: boolean,        // ← User toggles
  escalationLevel: number,  // ← Auto-incremented if missed
```

```
  escalationLevel: number   // ← Auto-incremented if missed
}

// incidents Collection
{
  _id: ObjectId,
  userEmail: string,        // ← Links to panicked user
  userName: string,         // ← From user record

  triggeredAt: Date,        // ← Auto-set when panic pressed

  location: {
    latitude: number,       // ← GPS captured by phone
    longitude: number,      // ← GPS captured by phone
    address: string         // ← Reverse-geocoded address
  },

  contactsNotified: [
    {
      contactId: ObjectId,
      contactName: string,
      notifiedAt: Date,     // ← Auto-set
      status: 'sent' | 'failed'
    }
  ],

  status: 'active' | 'disarmed' | 'escalated',
  disarmedAt: Date          // ← Auto-set when user disarms
}
```

KEY PRINCIPLE: Every field in the database either:

1. **User-entered data** (email, phone, name, documents, settings)
2. **Device-provided data** (GPS location, FCM token)
3. **Auto-calculated data** (dates, hashes, incident IDs)

NOTHING is hardcoded!

---

5. API ENDPOINTS EXPLAINED

How Frontend Calls Backend

```
Frontend (React Native) wants to add a contact:
```

```
STEP 1: User fills form
┌──────────────────────────────┐
| Contact Name: Maria Garcia   |
| Phone: +1(555)987-6543       |
| Email: maria@email.com       |
| Relationship: Mom            |
| [SAVE]                       |
└──────────────────────────────┘

STEP 2: Frontend makes POST request
const response = await fetch('/api/contacts', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${authToken}` // JWT token
  },
  body: JSON.stringify({
    name: 'Maria Garcia',
    phone: '+1(555)987-6543',
    email: 'maria@email.com',
    relationship: 'Mom'
  })
})

STEP 3: Backend receives request (server/src/routes/contacts.ts)
router.post('/contacts', (req, res) => {
  const userEmail = req.user.email // From JWT middleware
  const { name, phone, email, relationship } = req.body

  // Validate inputs
  if (!name || !phone) {
    return res.status(400).json({ error: 'Missing fields' })
  }

  // Create contact in MongoDB
  db.collection('contacts').insertOne({
    userEmail: userEmail,  // ← Only this user's contacts
    name: name,
    phone: phone,
    email: email,
    relationship: relationship,
    createdAt: new Date()
  })
```

```
  // Return success
  res.json({ success: true, message: 'Contact added' })
})

STEP 4: Frontend receives response
{
  "success": true,
  "message": "Contact added"
}

STEP 5: Frontend shows success & reloads contacts

┌──────────────────────────┐
│ ✅ Contact saved!        │
│                          │
│ Your Contacts:           │
│ • Maria Garcia (+555...) │ ← New!
│ [Add Another Contact]    │
└──────────────────────────┘
```

All Endpoints (No Hardcoding)

```
AUTH ENDPOINTS
==============
POST /api/auth/signup
  Input: email, password, fullName, phone (user enters)
  Stores: new user in MongoDB
  Returns: JWT token

POST /api/auth/login
  Input: email, password (user enters)
  Returns: JWT token (pulls from MongoDB)

POST /api/auth/logout
  Clears token from localStorage

CONTACTS ENDPOINTS
==================
GET /api/contacts
  Gets: All contacts for logged-in user
  Query: userEmail (from JWT)
  Returns: [{ name, phone, email, relationship }]

POST /api/contacts
  Input: name, phone, email, relationship (user enters)
```

```
  Stores: new contact in MongoDB

  Returns: success

PUT /api/contacts/:id
  Input: updated fields (user edits)
  Updates: existing contact in MongoDB
  Returns: success

DELETE /api/contacts/:id
  Deletes: specific contact from MongoDB
  Returns: success

DOCUMENTS ENDPOINTS
====================
GET /api/documents
  Gets: All documents for logged-in user
  Returns: [{ type, fileName, expiresAt, ... }]

POST /api/documents
  Input: file (user uploads), type, expiresAt (user enters)
  Encrypts: file with AES-256-GCM (client-side)
  Stores: encrypted file in MongoDB
  Returns: success

DELETE /api/documents/:id
  Deletes: document from MongoDB
  Returns: success

PANIC ENDPOINTS
===============
POST /api/panic/trigger
  Input: latitude, longitude, address (GPS from device)
  Gets: all contacts from MongoDB
  Sends: Firebase Cloud Message to each contact
  Stores: incident in MongoDB
  Returns: incident ID

POST /api/panic/disarm
  Input: safePhrase (user enters)
  Verifies: bcrypt compare with stored hash
  Updates: incident status to "disarmed"
  Returns: success
```

Every endpoint:

1. ✅ Gets user email from JWT token (no hardcoding)
2. ✅ Queries MongoDB for that user's data only
3. ✅ Returns user-specific data (no mixing users)
4. ✅ Updates user's own data (user input, not hardcoded)

---

6. FRONTEND TO BACKEND COMMUNICATION

Example: User Adds Emergency Contact

```
┌─────────────────────────────────┐
│ FRONTEND (React Native)         │
│                                 │
│ User fills form:                │
│ ├─ Name: "Maria Garcia"         │
│ ├─ Phone: "+1(555)987-6543"     │
│ └─ [SAVE]                       │
└─────────────────────────────────┘

          ↓
src/screens/AddContactScreen.tsx:

const handleAddContact = async () => {
  // 1. Get user's auth token from storage
  const authToken = localStorage.getItem('authToken')

  // 2. Prepare user's input data
  const contactData = {
    name: formData.name,           // "Maria Garcia"
    phone: formData.phone,         // "+1(555)987-6543"
    email: formData.email,
    relationship: formData.relationship
  }

  // 3. Send to backend
  const response = await fetch('/api/contacts', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${authToken}`
    },
    body: JSON.stringify(contactData)
  })
```

```
  // 4. Handle response
  if (response.ok) {
    showAlert('Contact saved!')
    reloadContacts() // Fetch updated list
  }
}

          ↓
┌──────────────────────────────────┐
│ BACKEND (Node.js + Express)      │
│                                  │
│ server/src/routes/contacts.ts    │
└──────────────────────────────────┘
          ↓
router.post('/contacts', authMiddleware, (req, res) => {
  // 1. Extract user from JWT token (middleware does this)
  const userEmail = req.user.email // "alex@email.com"

  // 2. Get data user sent
  const { name, phone, email, relationship } = req.body

  // 3. Validate
  if (!name || !phone) {
    return res.status(400).json({ error: 'Name & phone required' })
  }

  // 4. Create contact linked to THIS user
  const contact = {
    userEmail: userEmail,    // ← Links to Alex's account
    name: name,              // "Maria Garcia" (what user entered)
    phone: phone,            // "+1(555)987-6543" (what user entered)
    email: email,
    relationship: relationship,
    createdAt: new Date()
  }

  // 5. Insert into MongoDB
  db.collection('contacts').insertOne(contact)

  // 6. Return success
  res.json({
    success: true,
    message: 'Contact added',
    contactId: contact._id
```

```
  })
})

          ↓

┌──────────────────────────────────┐
│ MONGODB (Database)               │
│                                  │
│ Collection: contacts             │
│ New document:                    │
│ {                                │
│   _id: ObjectId(...),           │
│   userEmail: "alex@email.com",  │ ← Only Alex's
│   name: "Maria Garcia",          │ ← User's input
│   phone: "+1(555)987-6543",     │ ← User's input
│   email: "maria@email.com",     │
│   relationship: "Mom",           │
│   createdAt: 2026-05-09...      │
│ }                                │
└──────────────────────────────────┘

          ↓

Response goes back to Frontend:
{
  "success": true,
  "message": "Contact added",
  "contactId": "507f1f77bcf86cd799439013"
}

          ↓

┌──────────────────────────────────┐
│ FRONTEND (React Native)          │
│                                  │
│ ✅ Contact Saved!               │
│                                  │
│ Reload contact list:             │
│ • Maria Garcia (Mom)             │ ← New!
│ [Add Another Contact]            │
└──────────────────────────────────┘
```

Key Points:

1. ✅ User enters data (not hardcoded)
2. ✅ Frontend sends to backend
3. ✅ Backend verifies JWT (ensures right user)
4. ✅ Backend links data to user's email
5. ✅ Data stored in MongoDB (persistent)

6. ✅ User sees their data (not someone else's)

---

7. FIREBASE CLOUD MESSAGING FLOW

When User Presses 🚨 PANIC

```
STEP 1: User presses PANIC button (Frontend)

┌──────────────────────────────┐
│ [🚨 PANIC] ← Click           │
└──────────────────────────────┘


STEP 2: Frontend captures location & sends to backend
POST /api/panic/trigger
{
  "latitude": 38.5816,
  "longitude": -121.4944,
  "address": "123 Main St, Sacramento, CA"
}

STEP 3: Backend processes
├─ Verify user authenticated (JWT)
├─ Get user from MongoDB (alex@email.com)
├─ Get all contacts from MongoDB (query: userEmail = alex@email.com)
├─ Extract FCM tokens from each contact
│  (Contact has fcmToken because they signed up)
├─ Create incident record in MongoDB
└─ Send Firebase Cloud Message to each contact

STEP 4: Firebase Cloud Messaging

┌──────────────────────────────────────────────────────────┐
│ For each contact with FCM token:                         │
│                                                          │
│ admin.messaging().send({                                 │
│   notification: {                                        │
│     title: "🚨 Alex Chen NEEDS HELP!",                   │
│     body: "Location: 123 Main St, Sacramento, CA"        │
│   },                                                     │
│   data: {                                                │
│     incidentId: "507f1f77bcf86cd799439012",              │
│     userName: "Alex Chen",                               │
│     address: "123 Main St, Sacramento, CA",              │
│     latitude: "38.5816",                                 │
│     longitude: "-121.4944",                              │
```

```
│     longitude: -121.4944                                  │
│   },                                                      │
│   token: "eESTx2nRfFk:APA91bFvMg..." ← Contact's FCM token
│ })                                                        │
└───────────────────────────────────────────────────────────┘

STEP 5: Contact's device receives notification
(Even if app is closed!)

┌───────────────────────────────────────────────────────────┐
│ Maria's iPhone                                            │
│ 🚨 Alex Chen NEEDS HELP!                                  │
│ Location: 123 Main St, Sacramento, CA                     │
│ [TAP TO VIEW]                                             │
└───────────────────────────────────────────────────────────┘

STEP 6: Contact taps notification
├─ Opens PAN!C app
├─ Frontend fetches incident details from backend
├─ Shows: Alex's location, documents, emergency info
└─ Contact can call emergency services OR tap to help

STEP 7: Backend logs everything
MongoDB incident document:
{
  userEmail: "alex@email.com",
  userName: "Alex Chen",
  triggeredAt: 2026-05-09T14:30:00Z,
  location: {
    latitude: 38.5816,
    longitude: -121.4944,
    address: "123 Main St, Sacramento, CA"
  },
  contactsNotified: [
    {
      contactName: "Maria Garcia",
      status: "sent",
      notifiedAt: 2026-05-09T14:30:01Z
    }
  ],
  status: "active"
}
```

Why Firebase FCM (Not SMS):

- ✅ Instant (<100ms vs 1-2 seconds)

- ✅ Free (vs $0.01 per SMS)
- ✅ Works even if app is closed
- ✅ Works globally
- ✅ No phone number needed for contact to receive alert

---

8. REAL EXAMPLE: User Presses PANIC BUTTON

Let's trace the complete flow:

```
TIME: 2:30 PM - Alex is stopped by ICE agents

SECOND 1: Alex presses 🚨 PANIC
  Frontend: Gets GPS (38.5816, -121.4944)
  Frontend: Reverse-geocodes to "123 Main St, Sacramento, CA"
  Frontend: Calls POST /api/panic/trigger

SECOND 2: Backend receives request
  Backend: Verifies JWT token ✅
  Backend: Extracts userEmail: "alex@email.com"
  Backend: Queries MongoDB for all contacts where userEmail = "alex@email.com"
  Results: [
    { name: "Maria Garcia", fcmToken: "eESTx2nRfFk:APA91..." },
    { name: "Carlos Lopez", fcmToken: "fFTUy3oSgGl:APA92..." }
  ]

SECOND 3: Backend creates incident record
  MongoDB insertion:
  db.collection('incidents').insertOne({
    _id: ObjectId("123abc456def789"),
    userEmail: "alex@email.com",
    userName: "Alex Chen",
    triggeredAt: new Date(),
    location: { latitude: 38.5816, longitude: -121.4944, ... },
    contactsNotified: [
      { contactName: "Maria Garcia", status: "pending" },
      { contactName: "Carlos Lopez", status: "pending" }
    ],
    status: "active"
  })

SECOND 4: Backend sends Firebase Cloud Messages
  Message 1 to Maria:
```

```
  admin.messaging().send({

    notification: {
      title: "🚨 Alex Chen NEEDS HELP!",
      body: "Location: 123 Main St, Sacramento, CA"
    },
    token: "eESTx2nRfFk:APA91..."
  })
  ✅ Sent in <100ms

  Message 2 to Carlos:
  admin.messaging().send({
    notification: {
      title: "🚨 Alex Chen NEEDS HELP!",
      body: "Location: 123 Main St, Sacramento, CA"
    },
    token: "fFTUy3oSgGl:APA92..."
  })
  ✅ Sent in <100ms

SECOND 5: Backend returns response
  {
    "incidentId": "123abc456def789",
    "contactsNotified": 2,
    "status": "active"
  }

SECOND 6: Frontend receives response
  Shows: "Alert sent to 2 contacts"
  Displays: PANIC ACTIVE screen
  ├─ Flashing red header
  ├─ Audio playing: "HELP ICE AGENTS..."
  ├─ Location sharing
  ├─ Safe phrase input to disarm

SECOND 7: Maria's phone (400 miles away)
  🔔 Push notification arrives
  Title: "🚨 Alex Chen NEEDS HELP!"
  Body: "Location: 123 Main St, Sacramento, CA"

SECOND 8: Maria sees notification
  (Even if she was using another app!)
  Taps notification → Opens PAN!C

SECOND 9: Maria sees incident details
```

```
  ├─ Alex's name
  ├─ Location on map
  ├─ Documents (secure links)
  ├─ Legal rights summary
  └─ Call emergency services button

MINUTE 5: Alex is safe
  Enters safe phrase: "my_secret_phrase"
  Backend verifies with bcrypt hash ✅
  Updates MongoDB: status = "disarmed"
  Stops audio on Alex's phone
  Updates incident: disarmedAt = now

PERMANENT RECORD:
  MongoDB incident document shows:
  ├─ When panic was triggered
  ├─ Who was notified and when
  ├─ When it was disarmed
  └─ Complete timeline for later reference
```

---

9. WHY NOTHING IS HARDCODED

What "Hardcoded" Means

```javascript
// ❌ HARDCODED (Bad)
const emergencyContacts = [
  "Mom: +1(555)987-6543",
  "Dad: +1(555)123-4567",
  "Brother: +1(555)999-8888"
];
// Every user sees these same contacts!
// New user can't add their own
// Can't delete
// Not scalable

// ✅ DYNAMIC (Good)
db.collection('contacts').find({ userEmail: userEmail })
// Each user sees ONLY their contacts
// User can add/edit/delete
// Scales to millions of users
```

Every User Gets Different Data

```
User: Alex Chen
├─ Email: alex@email.com
├─ Contacts: Maria, Carlos
├─ Documents: Passport, Visa
└─ Check-In Interval: 30 minutes

User: Maria Garcia
├─ Email: maria@email.com
├─ Contacts: Alex, Mother, Lawyer
├─ Documents: Passport, ID, Birth Certificate
└─ Check-In Interval: 60 minutes

User: Brother Chen
├─ Email: brother@email.com
├─ Contacts: Alex, Maria, Mom
├─ Documents: Passport only
└─ Check-In Interval: Never (disabled)
```

Each user's data is:

1. ✅ Stored separately in MongoDB
2. ✅ Filtered by userEmail (no mixing)
3. ✅ User can modify (add/edit/delete)
4. ✅ Secure (JWT ensures right user)

---

10. TESTING THE COMPLETE FLOW LOCALLY

Test 1: Full Signup & Check Contact Was Stored

```bash
# Terminal 1: Start the app
npm run dev
# Should see:
# ✓ Frontend running at http://localhost:5173
# ✓ Backend running at http://localhost:3001
# ✓ MongoDB connected
```

```bash
```

```
# Terminal 2: Sign up a test user
# In browser: http://localhost:5173
# Step 1: Email: test1@email.com, Password: Test123!
# Step 2: Name: Alex Chen, Phone: +1(555)123-4567
# Step 3: Contact: Maria Garcia, +1(555)987-6543
# Step 4: Safe Phrase: mysecretphrase
# Click: CREATE ACCOUNT
```

bash

```
# Terminal 3: Check MongoDB
# Go to: https://cloud.mongodb.com
# Select your cluster
# Go to Collections
# Database: panic

# Check users collection:
# Should see 1 document with email: "test1@email.com"

# Check contacts collection:
# Should see 1 document with:
# - userEmail: "test1@email.com"
# - name: "Maria Garcia"
# - phone: "+1(555)987-6543"
```

Test 2: Verify Only This User's Contacts Show

bash

```
# Terminal 1: Still running
# Go to: http://localhost:5173/contacts

# Should see:
# • Maria Garcia (+555)987-6543

# Check browser console (F12 → Console):
# Network tab should show:
# GET /api/contacts → Response shows only Maria
```

Test 3: Add Another Contact Dynamically

```bash
# In app: Click [Add Contact]
# Fill form:
# Name: Carlos Lopez
# Phone: +1(555)999-8888
# Relationship: Lawyer
# [SAVE]

# Go to MongoDB:
# Check contacts collection
# Should now have 2 documents (Maria + Carlos)
# Both with userEmail: "test1@email.com"
```

Test 4: Sign Up Second User (Different Data)

```bash
# In browser: Go back to splash screen
# Click [Sign Up]
# Step 1: Email: test2@email.com, Password: Test456!
# Step 2: Name: Maria Garcia, Phone: +1(555)987-6543
# Step 3: Contact: Doctor Smith, +1(555)555-5555
# Step 4: Safe Phrase: mariassecret
# [CREATE ACCOUNT]

# Go to MongoDB:
# Check contacts collection
# Should now have 3 documents:
# 1. userEmail: "test1@email.com", name: "Maria Garcia"
# 2. userEmail: "test1@email.com", name: "Carlos Lopez"
# 3. userEmail: "test2@email.com", name: "Doctor Smith" ← Different user!
```

Test 5: Verify Data Separation (Most Important!)

```bash



# Terminal: Test that users only see their own data
curl -H "Authorization: Bearer [test1-token]" http://localhost:3001/api/contacts
# Returns: Maria Garcia, Carlos Lopez
```

```
curl -H "Authorization: Bearer [test2-token]" http://localhost:3001/api/contacts
# Returns: Doctor Smith ONLY

# ✅ PROOF: Data is not hardcoded, not shared, properly separated!
```

Test 6: Test Panic Flow

```bash
# User 1 (test1@email.com):
# ├─ Has contact: Maria Garcia (with FCM token)
# └─ Press 🚨 PANIC
#    └─ Backend queries MongoDB:
#       "Find all contacts where userEmail = test1@email.com"
#       └─ Returns: Maria Garcia
#          └─ Sends FCM to Maria's token
#             └─ Maria sees notification

# User 2 (test2@email.com):
# ├─ Has contact: Doctor Smith (different person)
# └─ Press 🚨 PANIC
#    └─ Backend queries MongoDB:
#       "Find all contacts where userEmail = test2@email.com"
#       └─ Returns: Doctor Smith (not Maria!)
#          └─ Sends FCM to Doctor's token
#             └─ Doctor sees notification (not Maria!)

# ✅ PROOF: Panic is user-specific, not hardcoded!
```

COMPLETE FLOW SUMMARY

```
┌─────────────────────────────────────────────────────────────┐
│                      USER'S PERSPECTIVE                     │
└─────────────────────────────────────────────────────────────┘


1. SIGNUP
   User enters: email, password, name, phone, contacts, documents
   ↓
   Backend stores in MongoDB (user-specific)

   ↓
   Frontend stores JWT token locally
```

```
2. HOME SCREEN
   Frontend queries: GET /api/contacts (via JWT)
   ↓
   Backend: Find contacts where userEmail = current user
   ↓
   Frontend displays: User's personal contacts
   ↓
   User sees: Only their own emergency contacts (not others')

3. ADD CONTACT
   User enters: name, phone, relationship
   ↓
   Frontend sends: POST /api/contacts
   ↓
   Backend: Stores in MongoDB linked to userEmail
   ↓
   MongoDB: New document created (user-specific)

4. UPLOAD DOCUMENT
   User selects: file, type, expiry date
   ↓
   Frontend: Encrypts with AES-256-GCM (client-side)
   ↓
   Frontend sends: POST /api/documents (encrypted bytes)
   ↓
   Backend: Stores encrypted data in MongoDB
   ↓
   MongoDB: Document only decryptable by user's device

5. PANIC SITUATION
   User presses: 🚨 PANIC
   ↓
   Frontend: Captures GPS, sends to backend
   ↓
   Backend:
     1. Verify user (JWT)
     2. Query MongoDB: Get all contacts for this user
     3. Extract FCM tokens
     4. Send Firebase messages to each contact
     5. Create incident record in MongoDB
   ↓
   Contacts receive: Push notifications
   ↓
   MongoDB: Incident logged (user-specific)
```

```
6. DISARM PANIC
   User enters: safe phrase
   ↓
   Backend: Verify bcrypt hash
   ↓
   MongoDB: Update incident status to "disarmed"
   ↓
   Frontend: Show success message


┌─────────────────────────────────────────────────────────────┐
│                    DATABASE PERSPECTIVE                     │
└─────────────────────────────────────────────────────────────┘


Every user's data is isolated:
├─ users[email="user1@email.com"] → User 1's profile
├─ contacts[userEmail="user1@email.com"] → User 1's contacts only
├─ documents[userEmail="user1@email.com"] → User 1's documents only
├─ incidents[userEmail="user1@email.com"] → User 1's emergencies only
│
├─ users[email="user2@email.com"] → User 2's profile
├─ contacts[userEmail="user2@email.com"] → User 2's contacts only
├─ documents[userEmail="user2@email.com"] → User 2's documents only
└─ incidents[userEmail="user2@email.com"] → User 2's emergencies only

ISOLATION KEY: Every query filters by userEmail (from JWT token)
```

KEY TAKEAWAYS

✅ What Makes This NOT Hardcoded:

1. **User Input**: Every piece of data comes from user (name, email, contacts, documents)
2. **Database Storage**: All data stored in MongoDB, not in code
3. **Data Isolation**: Each user's data separated by email (stored in JWT)
4. **Dynamic Queries**: Backend queries MongoDB based on current user, not hardcoded lists
5. **Scalability**: Works for 1 user or 1 million users - same code
6. **User Control**: Users can add, edit, delete their own data anytime
7. **Security**: JWT ensures only authenticated user can access their data

✅ What Makes This Production-Ready:

1. **Encryption**: Documents encrypted with AES-256-GCM before storage

1. **Encryption**: Documents encrypted with AES-256-GCM before storage
2. **Authentication**: JWT tokens with 7-day expiry
3. **Passwords**: Bcrypt hashed (10 rounds)
4. **Firebase**: Instant push notifications (<100ms)
5. **Logging**: Every action logged in MongoDB
6. **Error Handling**: Backend validates all inputs
7. **User Privacy**: Each user sees only their own data

This is a real, production-grade system that scales to millions of users! 🚀